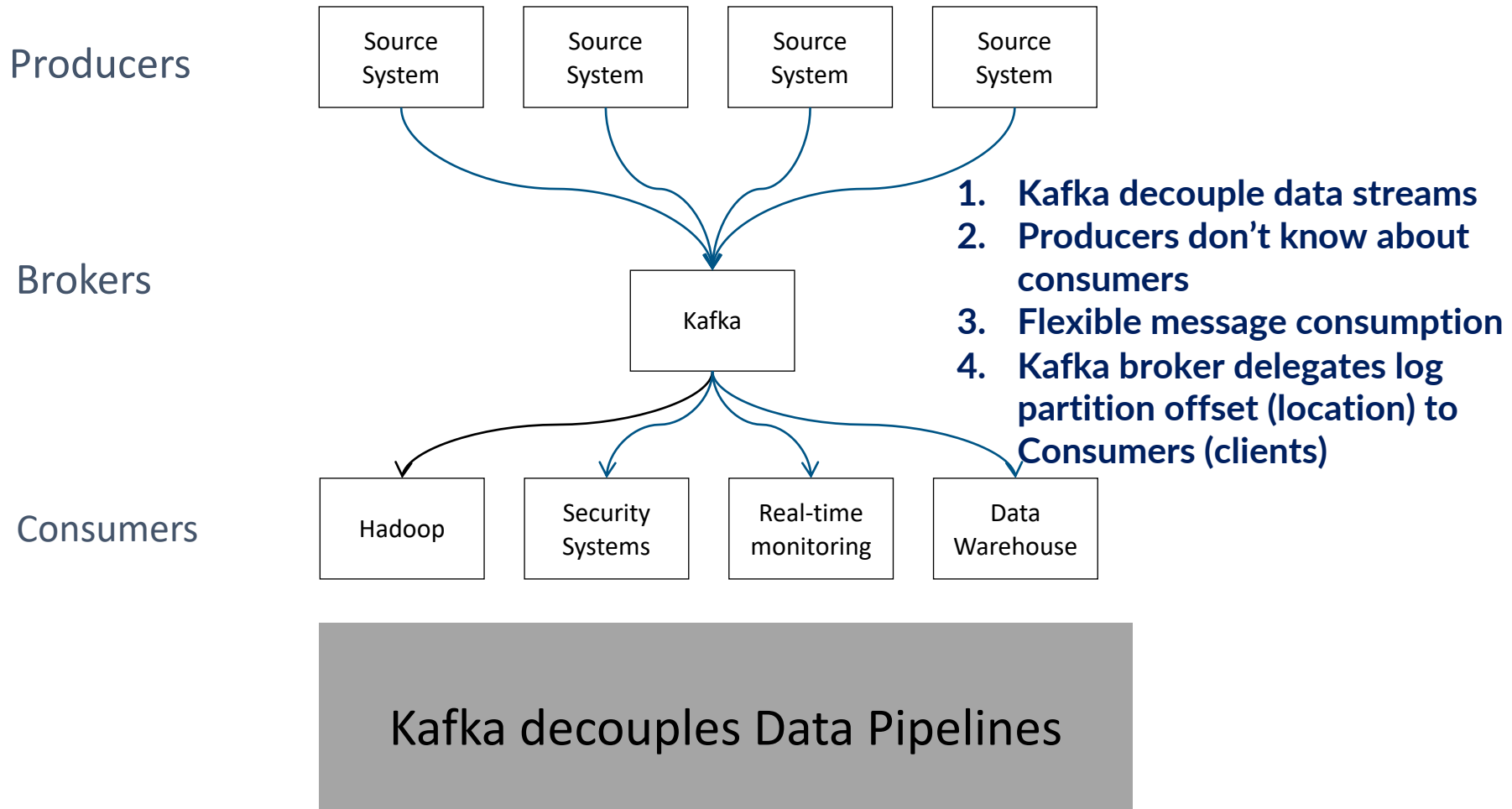




Hệ thống truyền thông điệp phân tán

ONE LOVE. ONE FUTURE.

Why Kafka



What is Kafka?

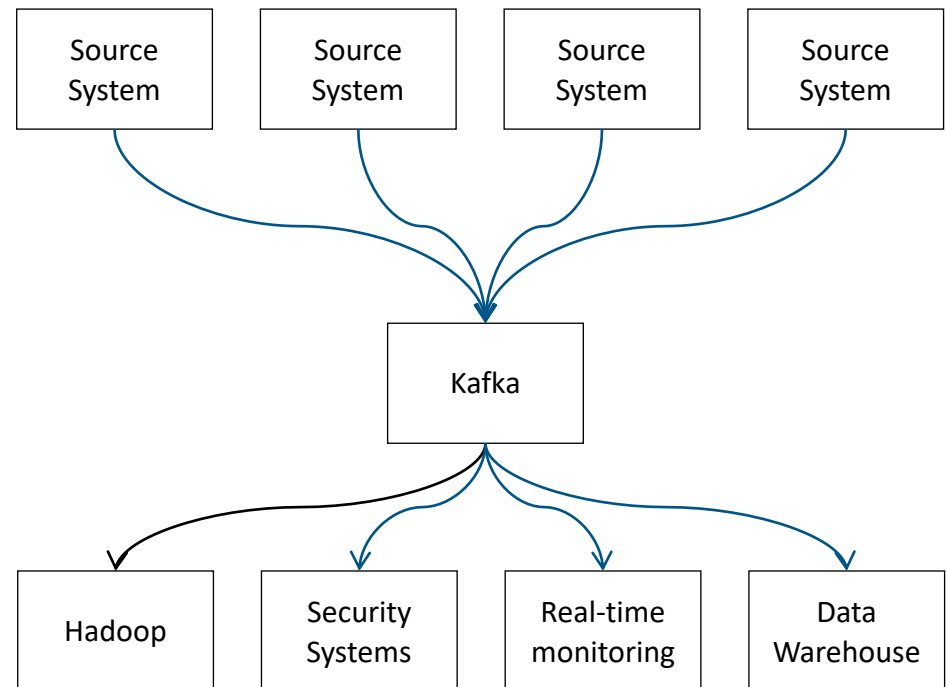
- Apache Kafka is a fast, scalable, durable, and fault-tolerant publish-subscribe messaging system
 - Publish and Subscribe to streams of records
 - Fault tolerant storage
 - Replicates Topic Log Partitions to multiple servers
 - Fast, efficient IO, batching, compression, and more
- Used to decouple data streams
- Kafka is often used instead of RabbitMQ
 - higher throughput, reliability and replication

Kafka possibility

- Build real-time streaming applications that react to streams
 - Feeding data to do real-time analytic systems
 - Transform, react, aggregate, join real-time data flows (eg. Metrics gathering)
 - Feed events to complex event processing engine
 - Feeding of high-latency daily or hourly data analysis into Spark, Hadoop, etc.
 - (eg. External commit log for distributed systems. Replicated data between nodes, re-sync for nodes to restore state)
 - Up to date dashboards and summaries

Concepts

Basic Kafka Concepts

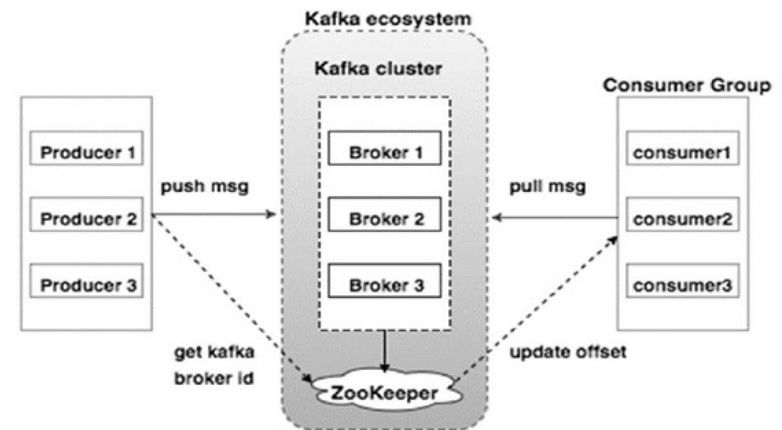


Key terminology

- Kafka maintains feeds of messages in categories called **topics**.
 - a stream of records (“/orders”, “/user-signups”), feed name
 - Log topic storage on disk
 - Partition / Segments (parts of Topic Log)
 - Records have a key (optional), value and timestamp; Immutable
- Processes that publish messages to a Kafka topic are called **producers**.
- Processes that subscribe to topics and process the feed of published messages are called **consumers**.
- Kafka is run as a cluster comprised of one or more servers each of which is called a **broker**.

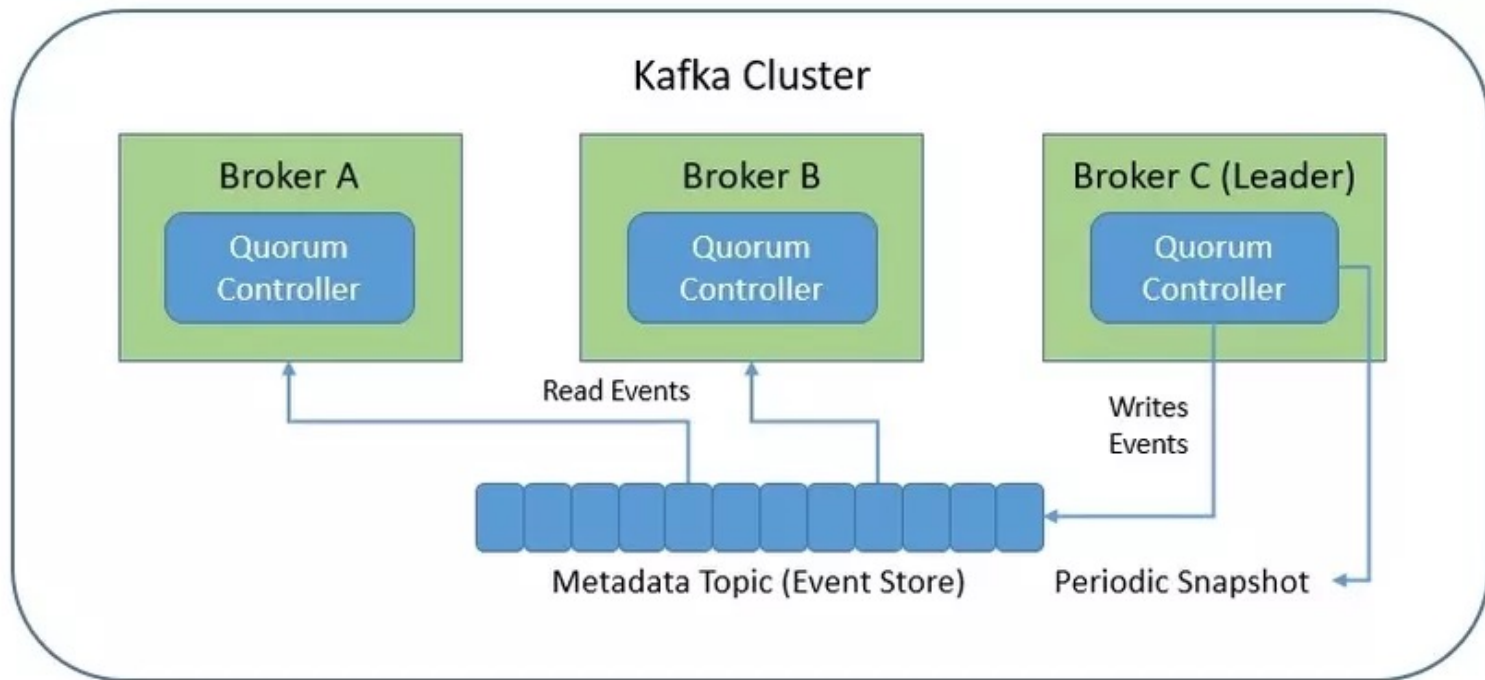
Kafka architecture

- Kafka cluster consists of multiple brokers and zookeeper
- Zookeeper provides in-sync view of Kafka Cluster configuration
 - Leadership election of Kafka Broker and Topic Partition pairs
 - manages service discovery for Kafka Brokers that form the cluster
- Zookeeper sends changes to Kafka
 - New Broker join, Broker died, etc.
 - Topic removed, Topic added, etc.

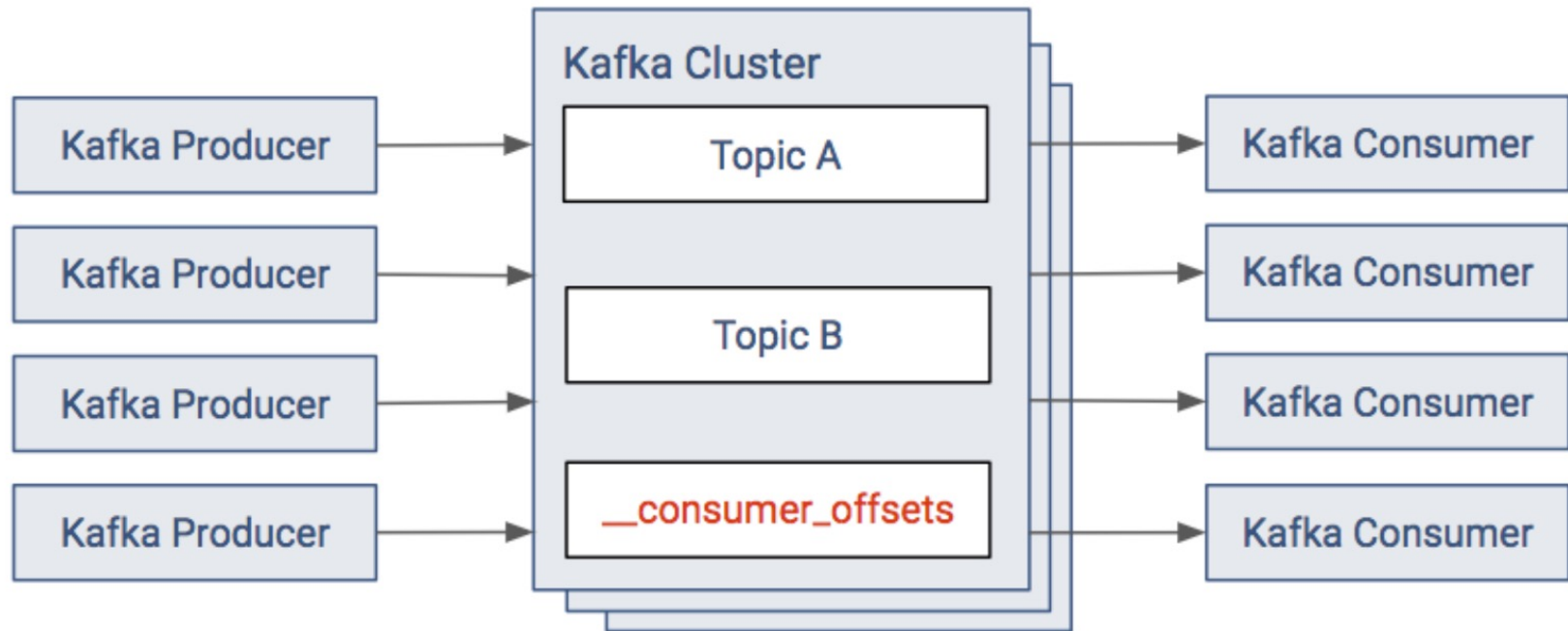


Apache Kafka Kraft

- The consensus protocol to remove Apache Kafka's dependency on ZooKeeper for metadata management



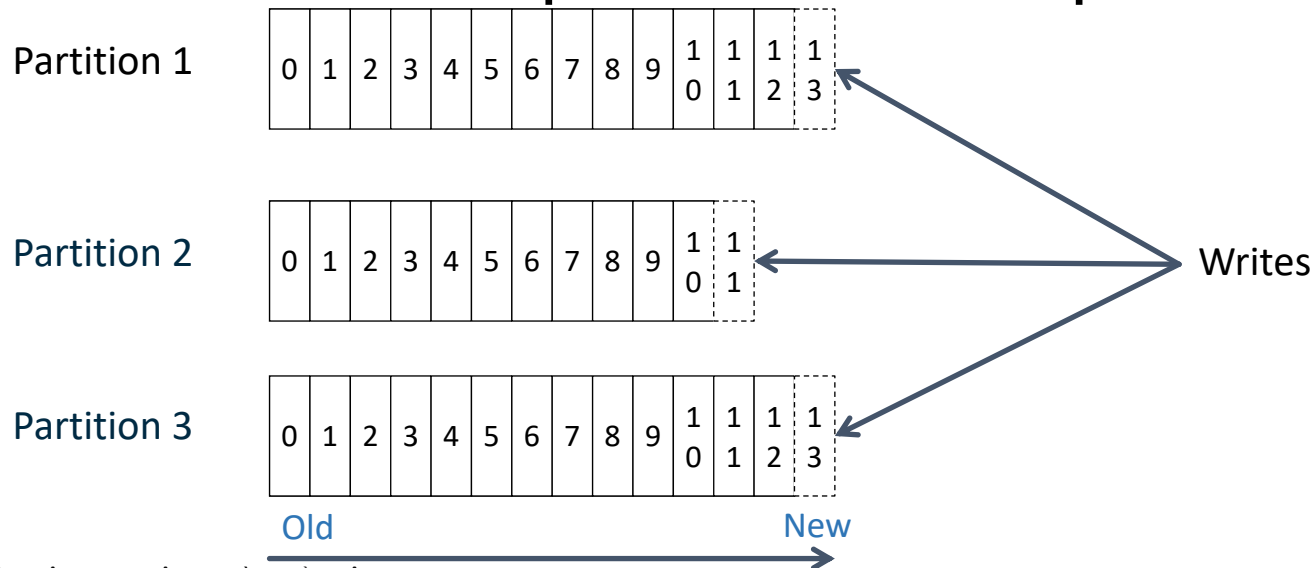
Topics, producers, and consumers



Kafka topics architecture

Topic partitions

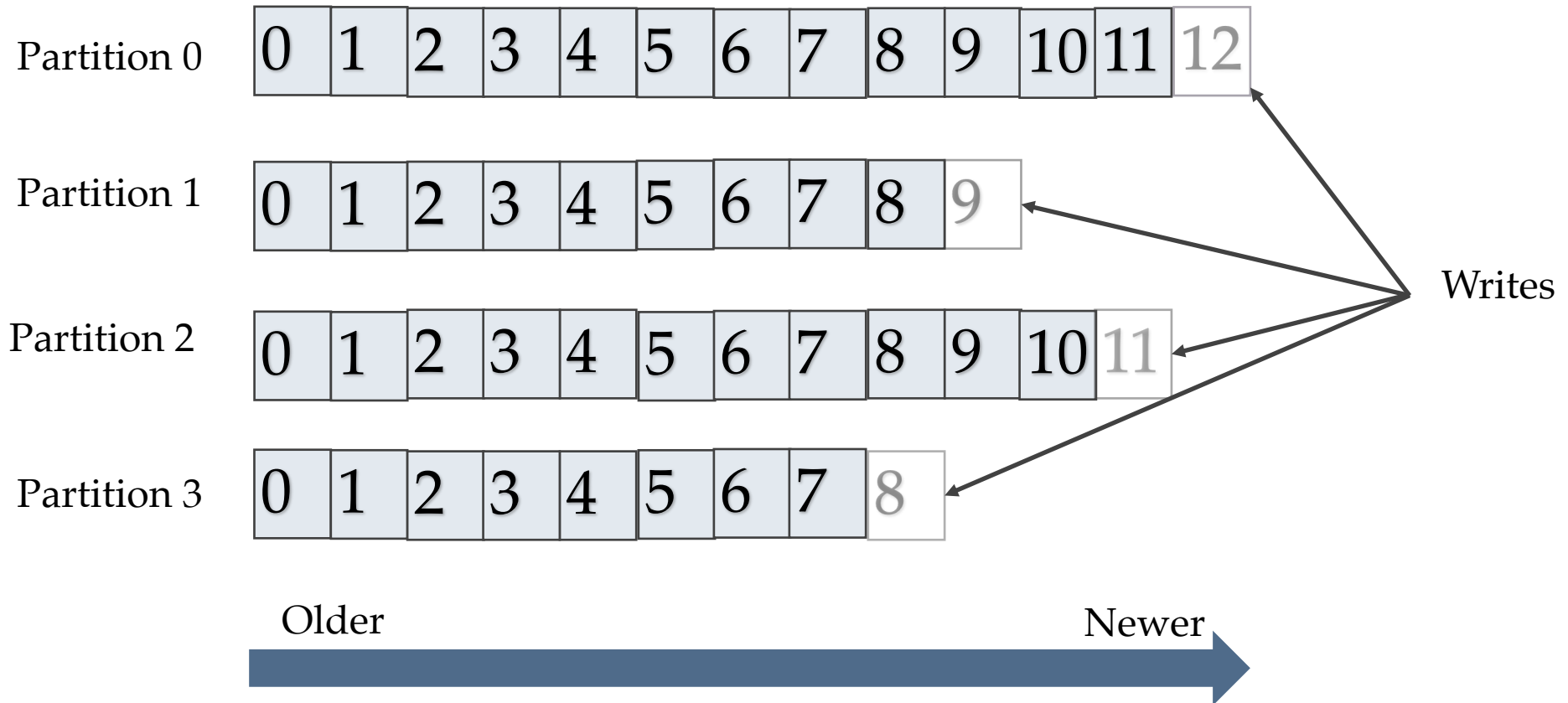
- Topics are broken up into partitions, decided usually by key of record
- Partitions are used to scale Kafka across many servers
 - Record sent to correct partition by key
- Partitions can be replicated to multiple brokers



Topic partition log

- Order is maintained only in a single partition
 - Partition is ordered, immutable sequence of records that is continually appended to—a **structured commit log**
- Records in partitions are assigned sequential id number called the offset

Kafka topic partitions layout



Kafka partition replication

- Each partition has leader server and zero or more follower servers
 - Leader handles all read and write requests for partition
 - Followers replicate leader
 - An follower that is in-sync is called an ISR (in-sync replica)
 - If a partition leader fails, one ISR is chosen as new leader
- Each partition can be replicated across a configurable number of Kafka servers
 - Used for fault tolerance

Kafka replication to partition (1)

Record is considered "committed" when all ISRs for partition wrote to their log.

Only committed records are readable from consumer

Client Producer

Leader **Red**
Follower **Blue**

1) Write record

2) Replicate
record

2) Replicate
record

Partition 0

Partition 1

Partition 2

Partition 3

Partition 4

Partition 0

Partition 1

Partition 2

Partition 3

Partition 4

Partition 0

Partition 1

Partition 2

Partition 3

Partition 4

Kafka replication to partitions (2)

Another partition can be owned by another leader on another Kafka broker

Client Producer

Leader **Red**
Follower **Blue**

1) Write record

2) Replicate record

2) Replicate record

- Message ordering
 - Messages sent by a producer to a particular topic partition will be appended in the order they are sent
 - A consumer instance sees messages in the order they are stored in the log
- Minimum available ISR
 - Can also be configured
 - Error is returned if enough replicas are not available to replicate data

Kafka record retention

- Kafka cluster retains all published records
 - Time based – configurable retention period
 - Size based - configurable based on size
 - Compaction - keeps latest record
- It is available for consumption until discarded by time, size or compaction
- Consumption speed not impacted by size

Durable writes

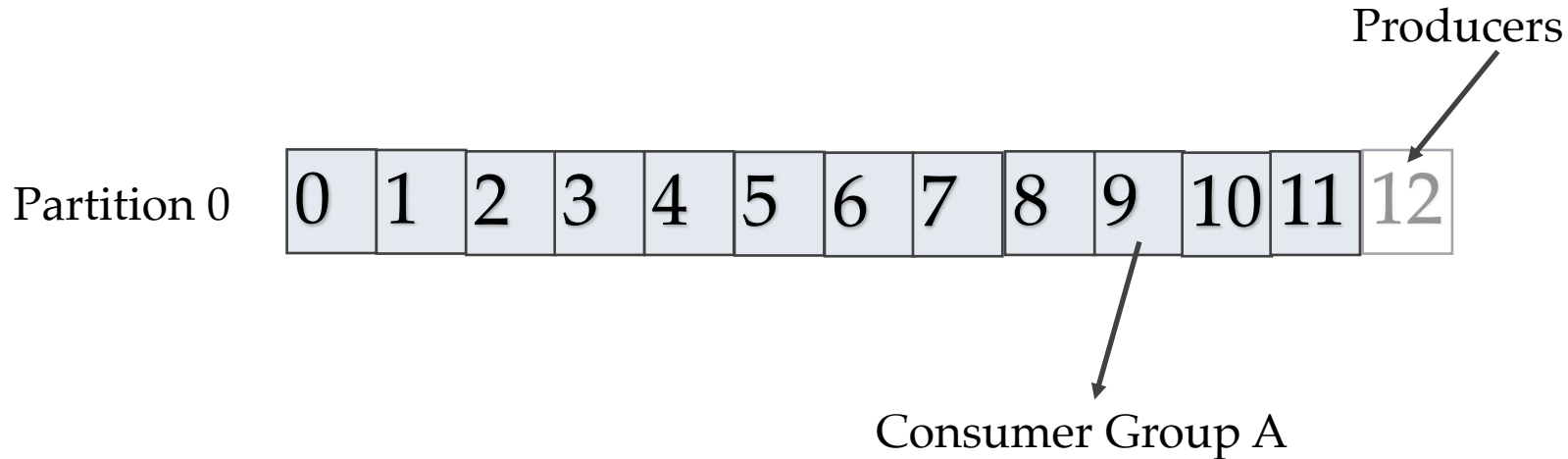
- Producers can choose to trade throughput for durability of writes:
- Note: throughput can also be raised with more brokers...

Durability	Behaviour	Per Event Latency
Highest	ACK all ISR's have received	Highest
Medium	ACK once the leader has received	Medium
Lowest	No ACKs required	Lowest

Producers

- Producers publish to a topic of their choosing (push)
 - Producer(s) append records at end of topic logs
- Load can be distributed in number of partitions
 - Typically, by “round-robin”
 - Can also do “semantic partitioning” based on a key in the message
 - Example have all the events of a certain 'employeeid' go to same partition
 - Important: **Producer picks partition**
- All nodes can answer metadata requests about
 - Which servers are alive
 - Where leaders are for the partitions of a topic

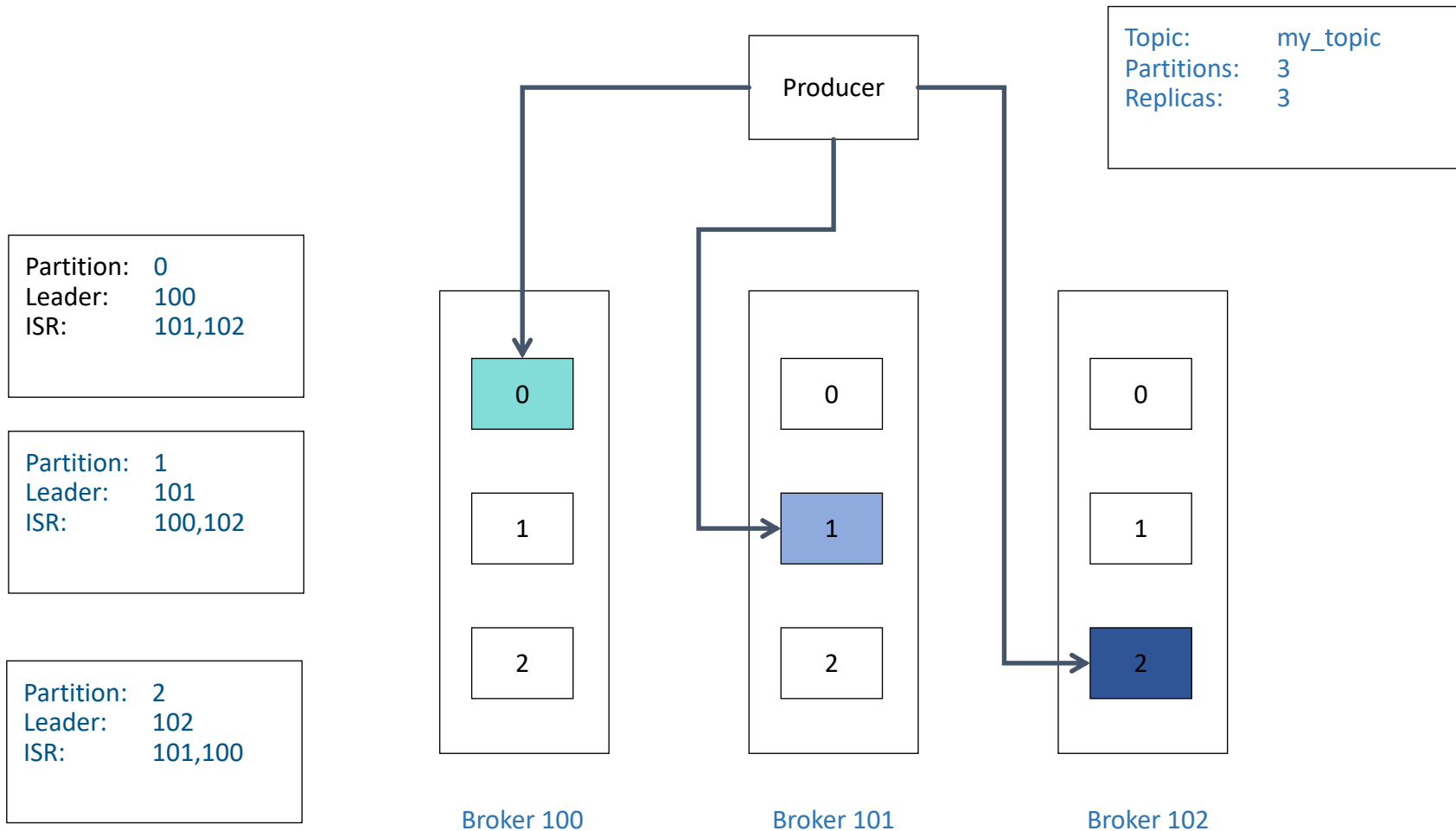
Kafka producers and consumers



Producers are writing at Offset 12

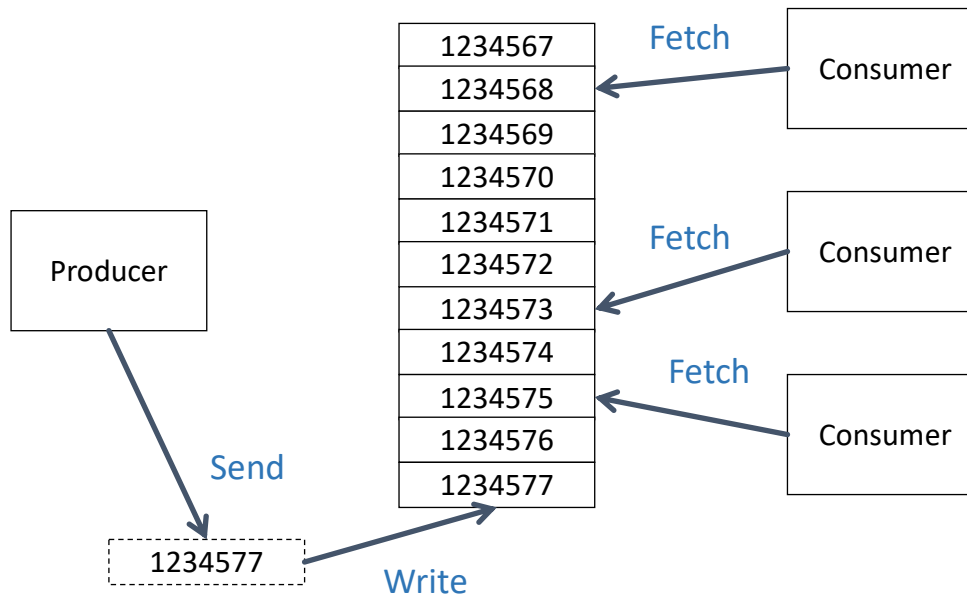
Consumer Group A is Reading from Offset 9.

Producer – Load balancing and ISR



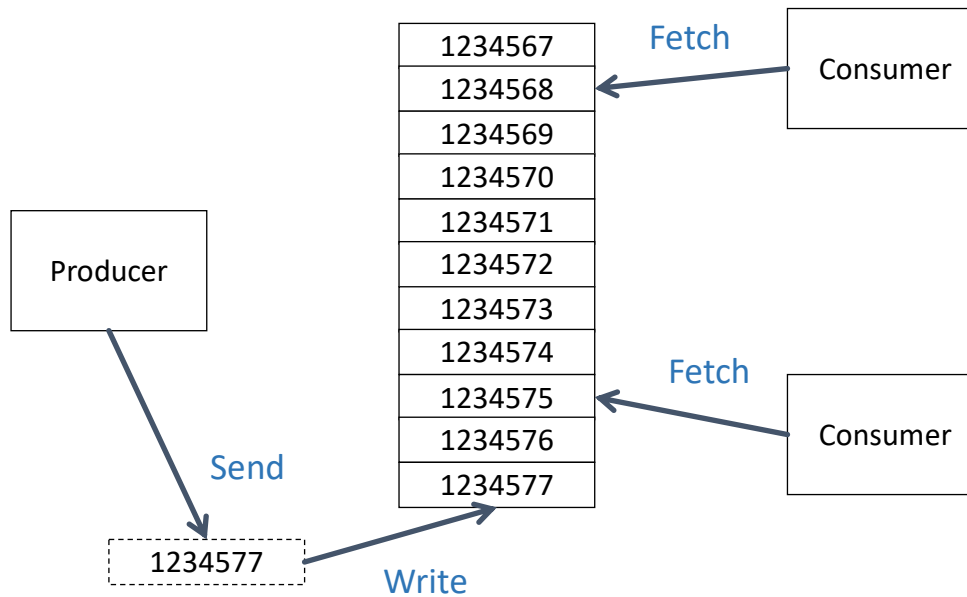
Consumer (1)

- Multiple Consumers can read from the same topic
- Each Consumer is responsible for managing it's own offset
- Messages stay on Kafka...they are not removed after they are consumed



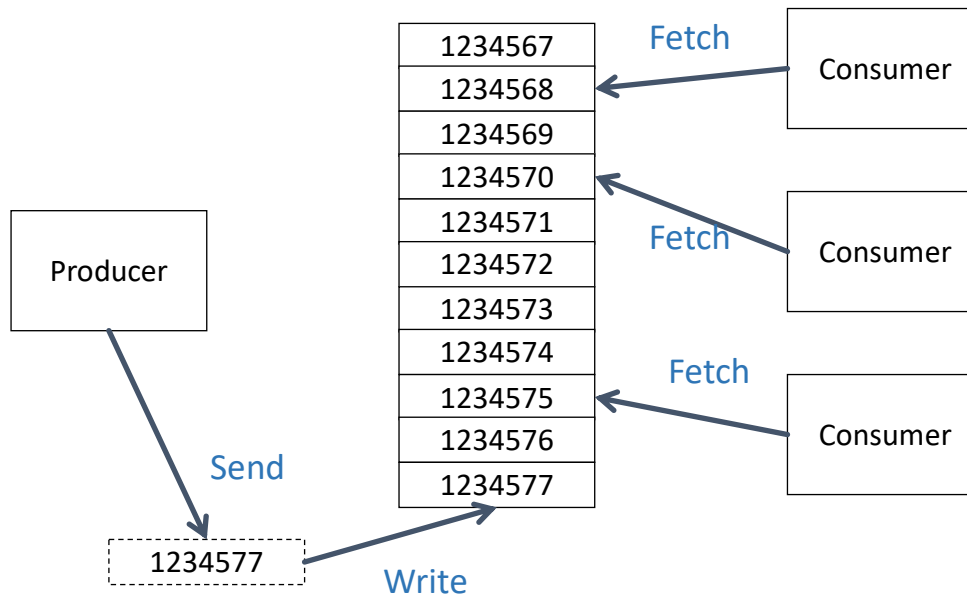
Consumer (2)

- Consumers can go away



Consumer (3)

- And then come back



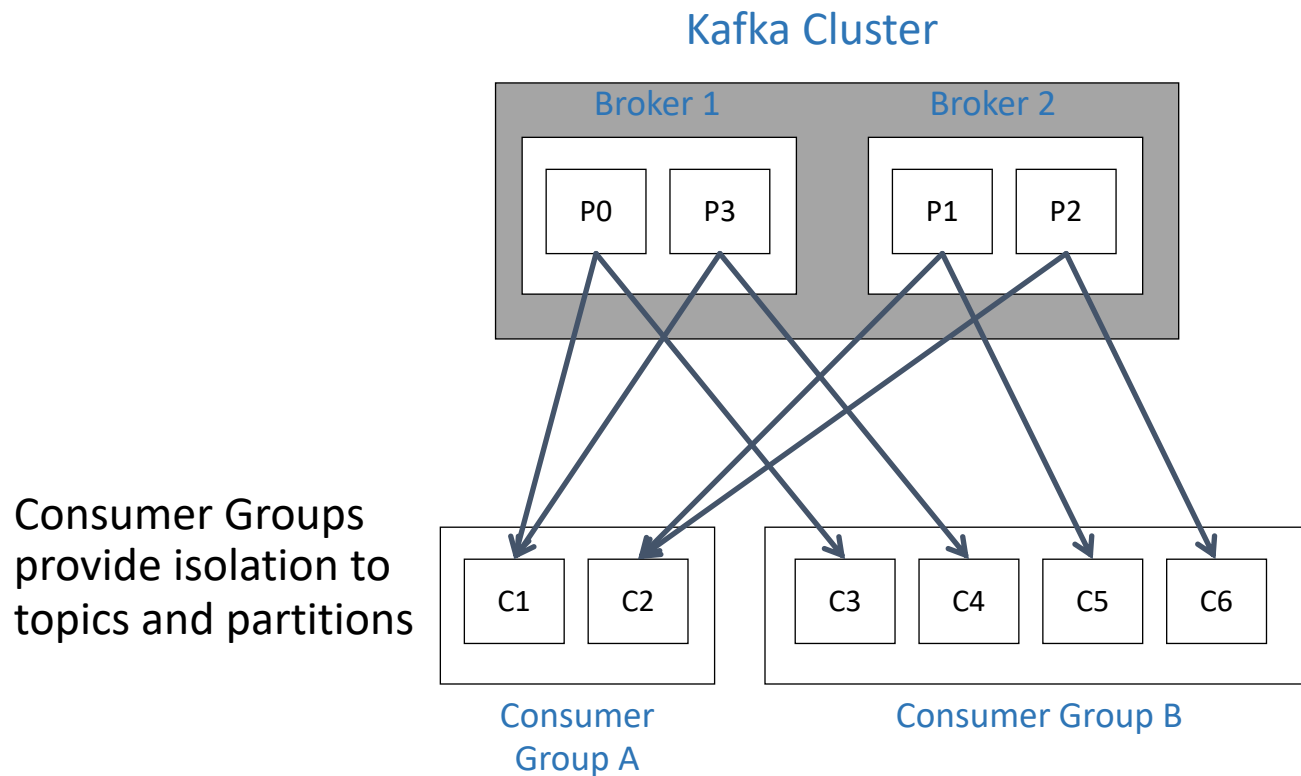
Consumer group

- Consumers are grouped into a consumer group
 - Consumer group has a unique id
 - Each consumer group is a subscriber
 - Each consumer group maintains its own offset
- Multiple subscribers = multiple consumer groups
 - Each has different function
 - One might deliver records to microservices while another is streaming records to Hadoop
- A record is delivered to one consumer in a consumer Group
 - Each consumer in consumer groups takes records
 - Only one consumer in group gets same record
 - Consumers in Consumer Group load balance record consumption

Common consumer group patterns

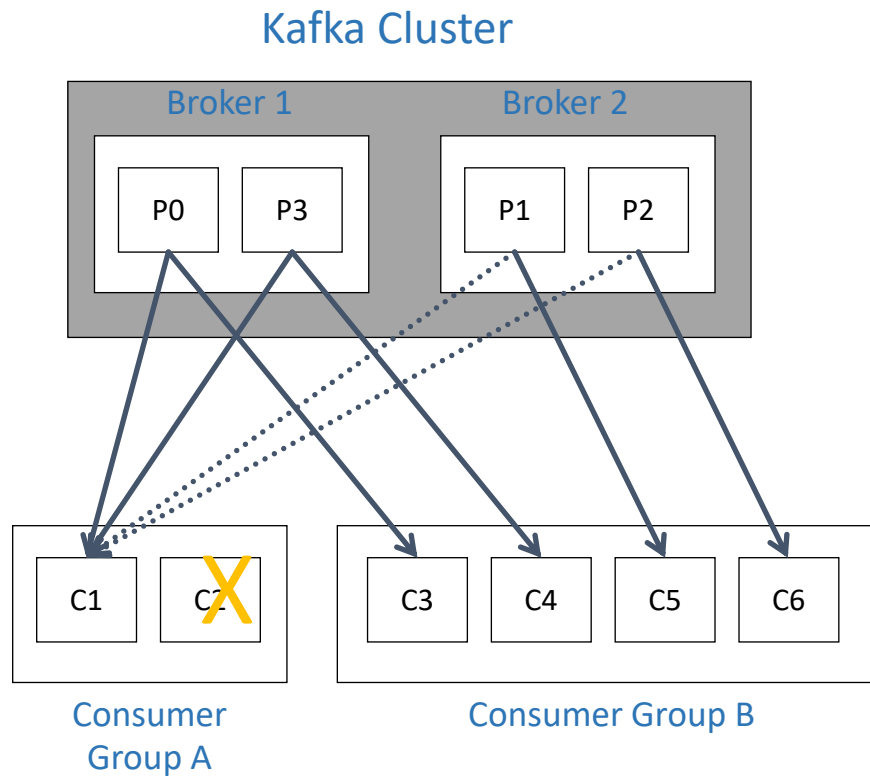
- All consumer instances in one group
 - Acts like a traditional queue with load balancing
- All consumer instances in different groups
 - All messages are broadcast to all consumer instances
- “Logical subscriber” – Many consumer instances in a group
 - Consumers are added for scalability and fault tolerance
 - Each consumer instance reads from one or more partitions for a topic

Consumer - Groups



Consumer - Groups

Can rebalance themselves



Kafka consumer load share

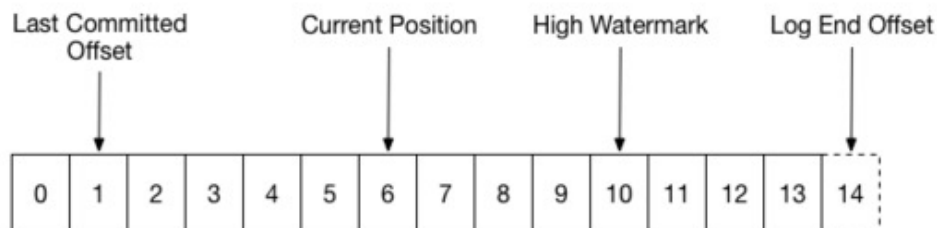
- Consumer membership in Consumer Group is handled by the Kafka protocol dynamically
 - If new Consumers join Consumer group, it gets a share of partitions
 - If Consumer dies, its partitions are split among remaining live Consumers in Consumer Group

Kafka consumer failover

- Consumers notify broker when it successfully processed a record
 - advances offset (“__consumer_offset”)
- If Consumer fails before sending commit offset to Kafka broker,
 - Different Consumer can continue from the last committed offset
 - Some Kafka records could be reprocessed
 - At least once behavior
 - Messages should be idempotent (immutable)

What can be consumed

- "Log end offset" is offset of last record written to log partition and where Producers write to next
- "High watermark" is offset of last record successfully replicated to all partition's followers
- Consumer only reads up to "high watermark"
- Consumer can't read un-replicated data



Consumer to partition cardinality

- Only a single Consumer from the same Consumer Group can access a single Partition
- If Consumers in a group exceeds Partition count:
 - Some Consumers remain idle; can be used for failover
- If more Partitions than Consumer Group instances,
 - Some Consumers will read from more than one partition

Kafka brokers

- Kafka Cluster is made up of multiple Kafka Brokers
 - Each Broker has an ID (number)
 - Start with at least three brokers, cluster can have, 10, 100, 1000 brokers if needed
- Brokers contain topic log partitions
- Connecting to one broker bootstraps client to entire cluster

Kafka scale and speed

- How can Kafka scale if multiple producers and consumers read/write to same Kafka Topic log?
 - Writes fast: Sequential writes to filesystem are fast (700 MB or more a second)
- Scales writes and reads by sharding:
 - Topic logs into partitions (parts of a topic log) in different machines/different disks
 - Multiple Producers can write to different Partitions of the same Topic
 - Multiple Consumers Groups can read from different partitions efficiently

Why is Kafka popular?

- Distributed Scalability
 - Architecture designed for scalability across multiple servers or clusters.
- High Throughput, Low Latency
 - Processes and transmits messages with high throughput and minimal latency for real-time data streaming.
- Fault Tolerance
 - Ensures reliability by replicating data across multiple brokers, allowing continued operation during failures.
- Durability and Persistence
 - Persists messages on disk, ensuring data integrity even in the event of system failures.
- Versatility
 - Serves multiple use cases, including event streaming, log aggregation, data integration, and messaging.

Why is Kafka popular?

- Decoupling Producers and Consumers
 - Provides flexibility in building distributed systems by decoupling data producers from consumers.
- Strong Community Support
 - Benefits from an active open-source community, ensuring stability and ongoing development.
- Seamless Integration with Big Data Ecosystem
 - Integrates well with Hadoop, Spark, and other components of the big data ecosystem.
- Real-time Streaming Analytics
 - Well-suited for real-time analytics, enabling businesses to gain insights and react to events as they happen.
- Comprehensive APIs
 - Offers APIs for various programming languages, making it accessible for developers in different environments.
- Wide Industry Adoption
 - Popular across industries like finance, telecommunications, and retail.